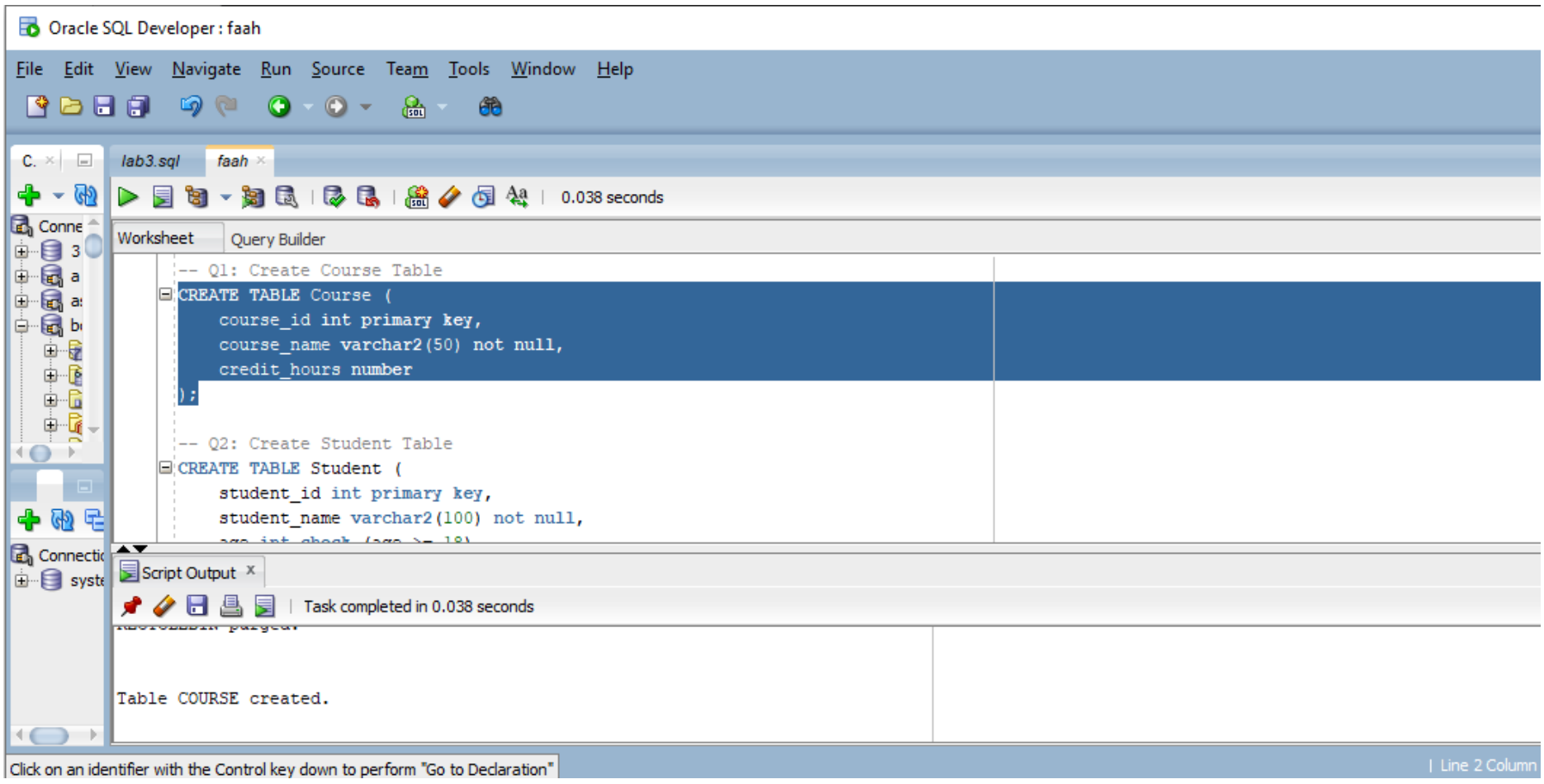
DBS Lab Task 3

Q1. Create a table `Course` with:

- course_id (Primary Key)
- course_name (NOT NULL)
- credit_hours (NUMBER)

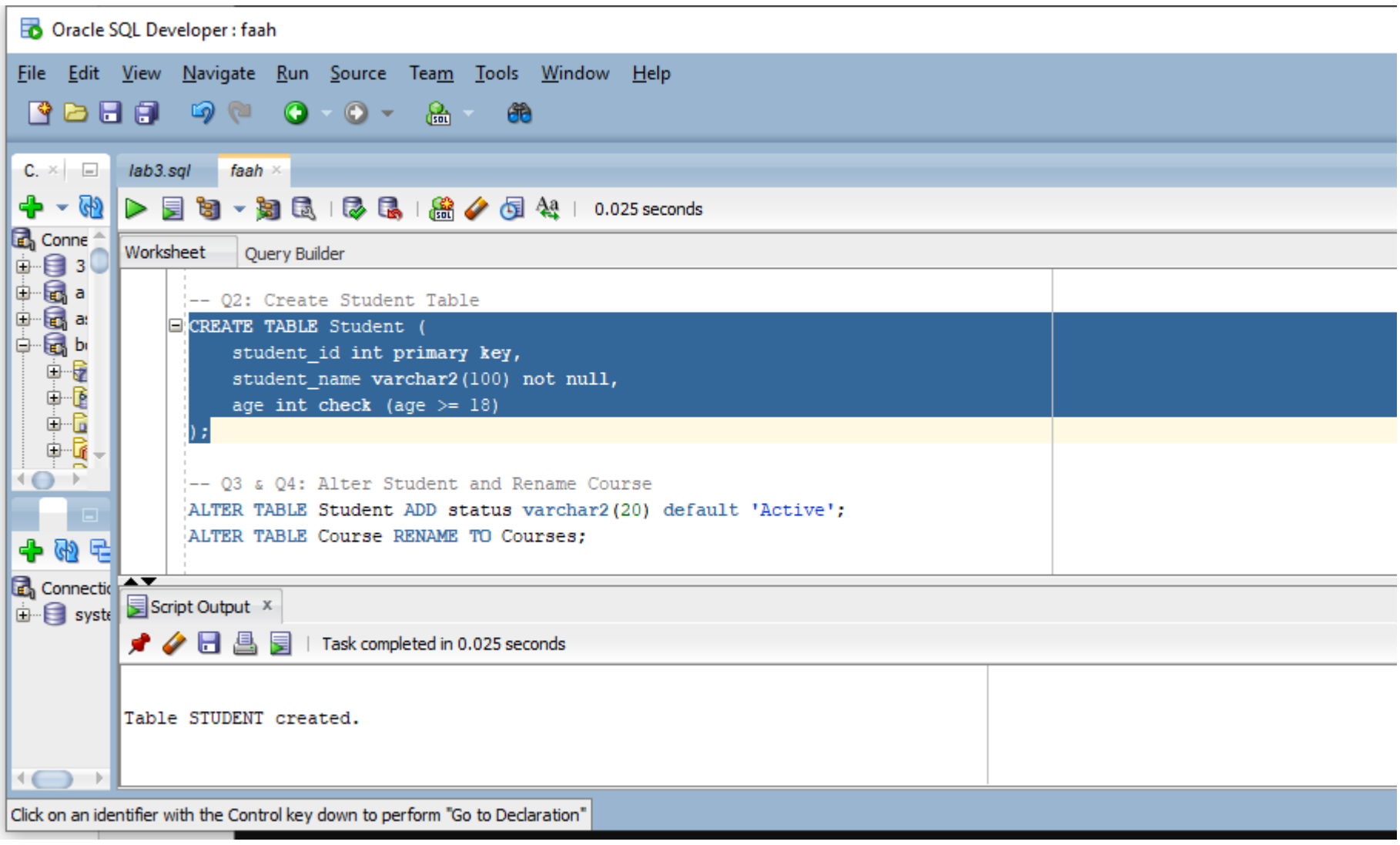
```
CREATE TABLE Course (
  course_id int primary key,
  course_name varchar2(50) not null,
  credit_hours number
);
```



Q2. Create a table `Student` with:

- student_id (Primary Key)
- student_name (NOT NULL)
- age (CHECK age >= 18)

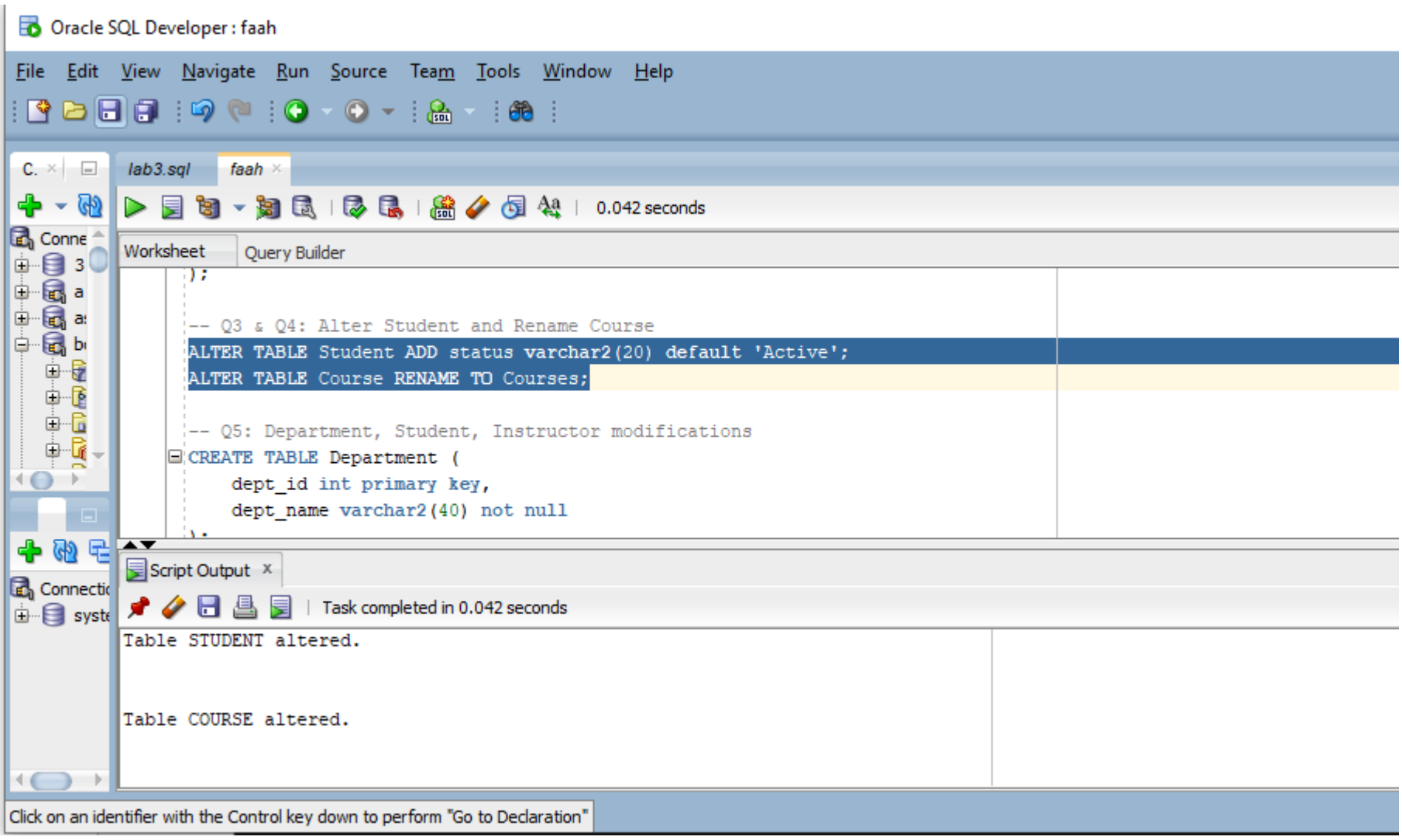
```
CREATE TABLE Student (
  student_id int primary key,
  student_name varchar2(100) not null,
  age int check (age >= 18)
);
```



Q3 & 4. USE STUDENT table with a column `status` having DEFAULT value 'Active'. Rename table `Course` to `Courses`.

ALTER TABLE Student ADD status varchar2(20) default 'Active';

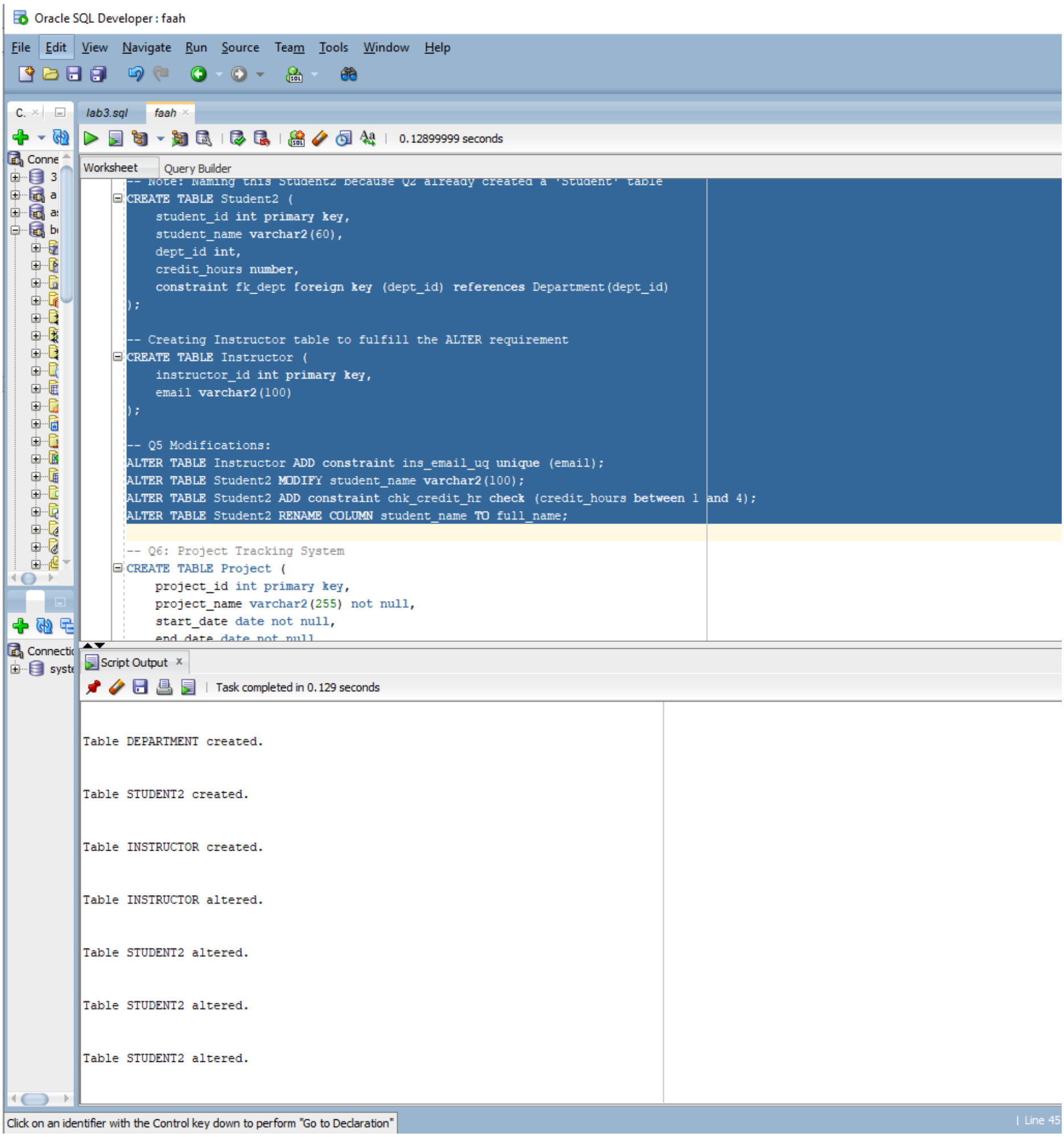
ALTER TABLE Course RENAME TO Courses;



Q5. Create a **Department** table containing `dept_id` (Primary Key) and `dept_name`. Then create a **Student** table containing `student_id` (Primary Key), `student_name`, `dept_id` (Foreign Key referencing Department), and `credit_hours`.

After creating the tables, perform the following modifications:
add a **UNIQUE constraint** on the `email` column in the **Instructor** table using `ALTER`, modify the `student_name` column to `VARCHAR2(100)`, add a **CHECK constraint** so `credit_hours` must be between **1 and 4**, and rename the column `student_name` to `full_name`.

```
CREATE TABLE Department (
   dept_id int primary key,
   dept_name varchar2(40) not null
);
-- Note: Naming this Student2 because Q2 already created a 'Student' table
CREATE TABLE Student2 (
   student_id int primary key,
   student_name varchar2(60),
   dept_id int,
   credit_hours number,
   constraint fk_dept foreign key (dept_id) references Department(dept_id)
);
-- Creating Instructor table to fulfill the ALTER requirement
CREATE TABLE Instructor (
   instructor_id int primary key,
   email varchar2(100)
);
-- Q5 Modifications:
ALTER TABLE Instructor ADD constraint ins_email_uq unique (email);
ALTER TABLE Student2 MODIFY student_name varchar2(100);
ALTER TABLE Student2 ADD constraint chk_credit_hr check (credit_hours between 1 and 4);
ALTER TABLE Student2 RENAME COLUMN student_name TO full_name;
```



Q6. You are designing a **Project Tracking System**.
Requirements:

- Table `Project` with columns: project_id, project_name, start_date, end_date, budget
- budget must be greater than 10000

end_date must always be after start_date

Task: Write the CREATE TABLE statement with all required constraints.

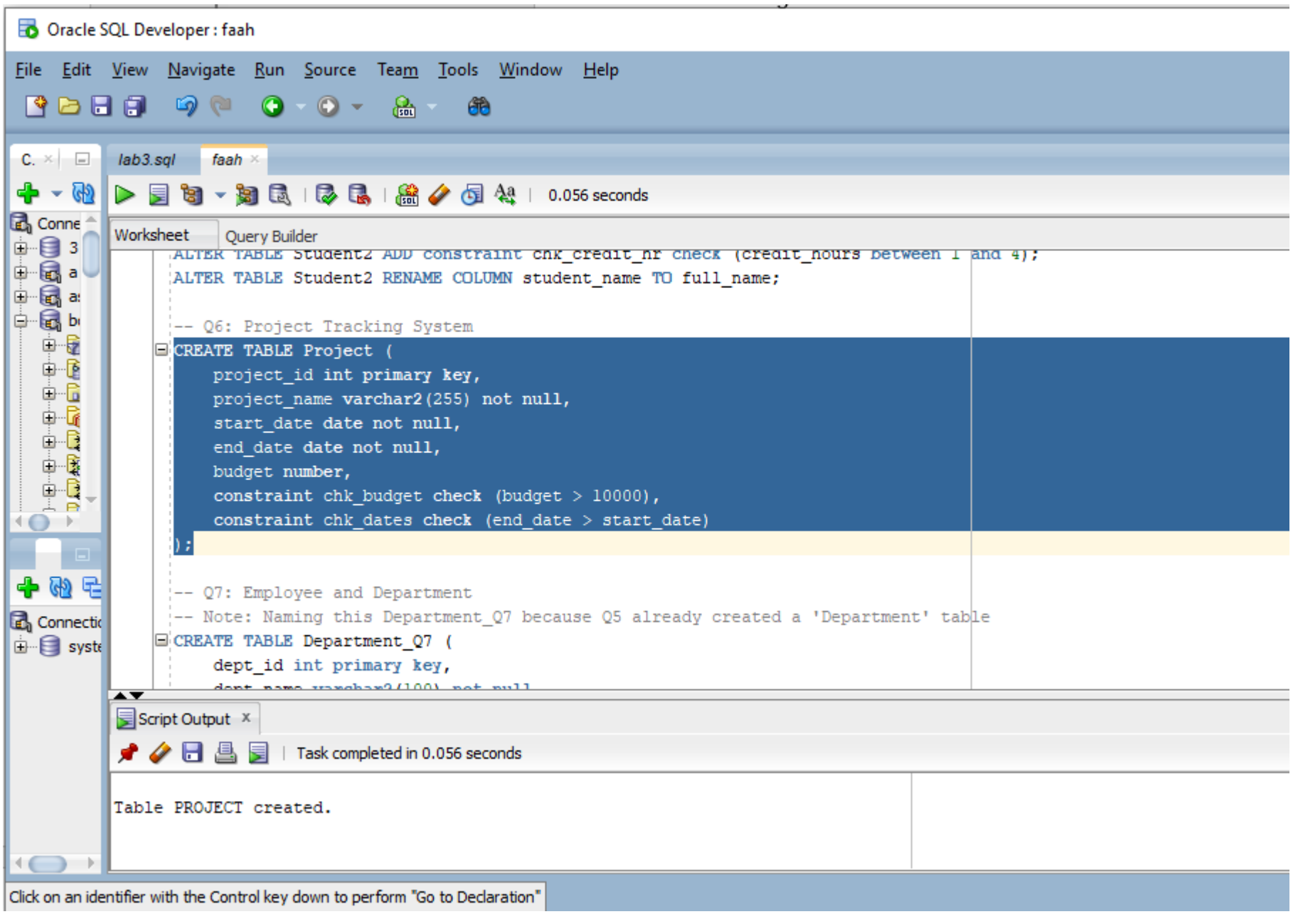
CREATE TABLE Project (

project_id int primary key,

project_name varchar2(255) not null,

start_date date not null,

```
  end_date date not null,
  budget number,
  constraint chk_budget check (budget > 10000),
  constraint chk_dates check (end_date > start_date)
);
```



Q7. A company tracks employees and departments.

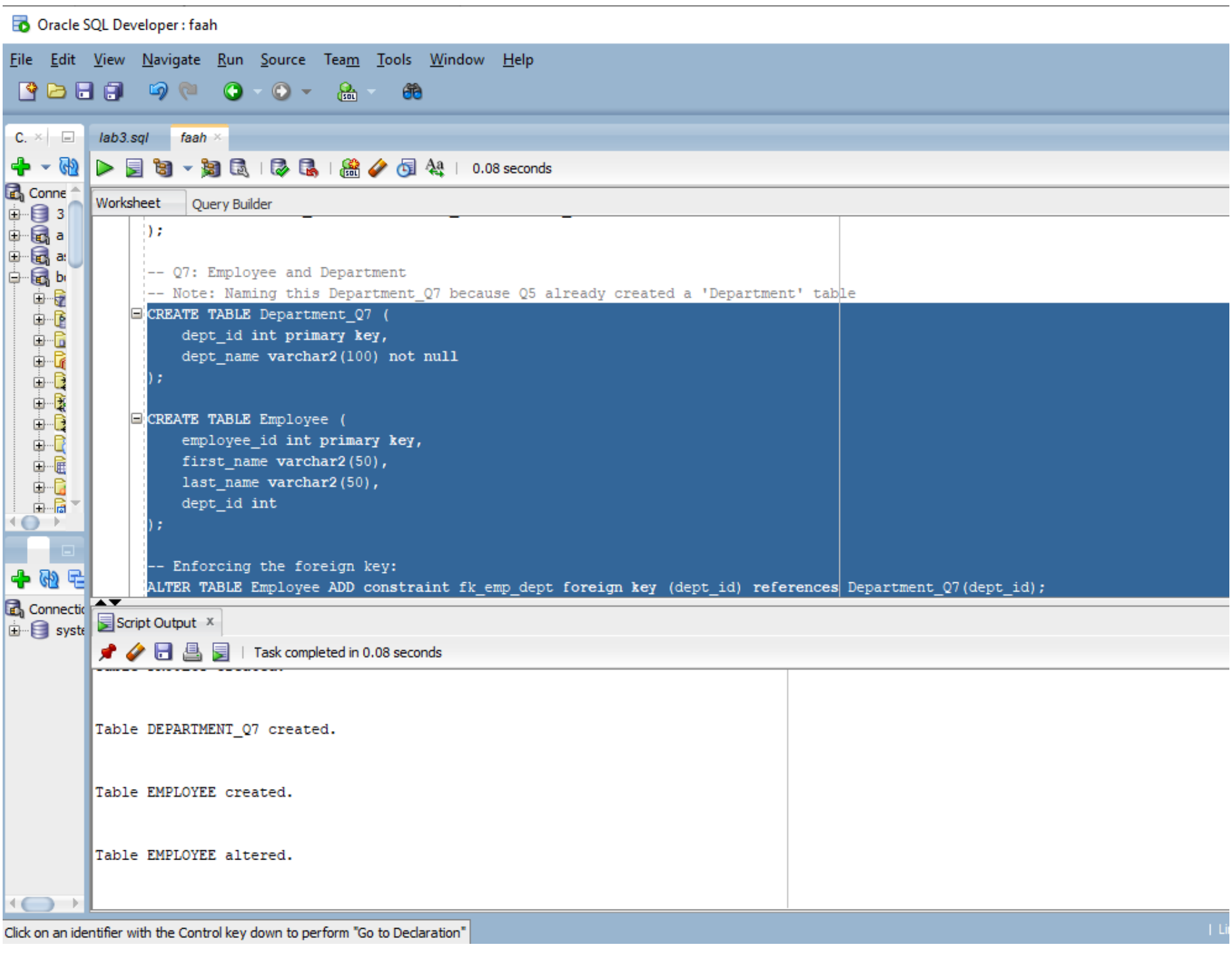
- Table `Employee` references `Department`.
- You need to **prevent assigning an employee to a non-existent department**.

Task: Write ALTER TABLE commands to enforce this foreign key.
Also, explain what happens if someone tries to delete a department that still has employees.

```
CREATE TABLE Department_Q7 (
  dept_id int primary key,
  dept_name varchar2(100) not null
);

CREATE TABLE Employee (
  employee_id int primary key,
  first_name varchar2(50),
  last_name varchar2(50),
```

```
  dept_id int
);
-- Enforcing the foreign key:
ALTER TABLE Employee ADD constraint fk_emp_dept foreign key (dept_id) references Department_Q7(dept_id);
```



Q8. A **startup company** wants to create a table `Product` with the following:

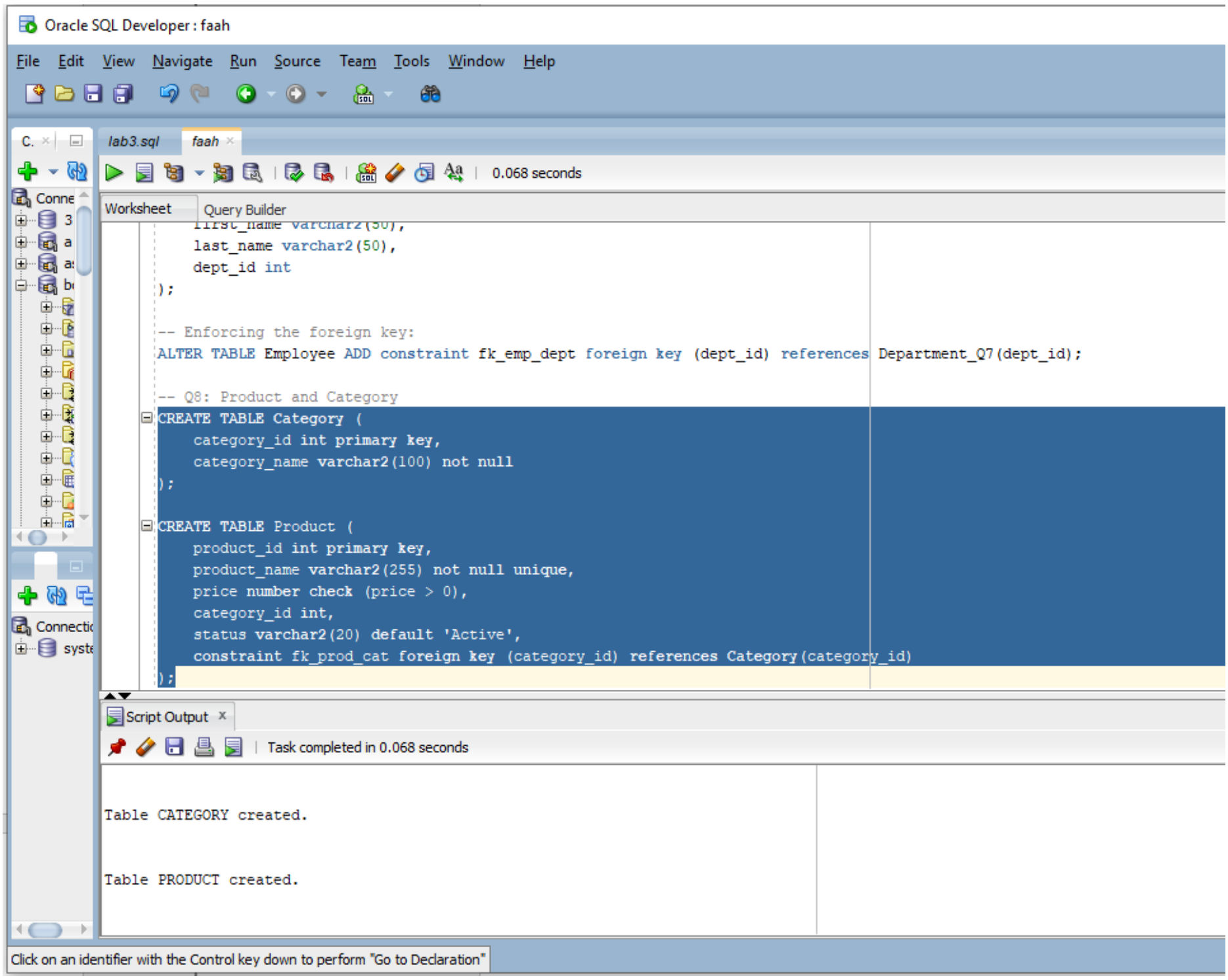
- product_id (PK)
- product_name (unique, not null)
- price (check > 0)
- category_id (FK to Category table)
- status (default = 'Active')

Task: Write the full CREATE TABLE statement.

```
CREATE TABLE Category (
  category_id int primary key,
  category_name varchar2(100) not null
);

CREATE TABLE Product (
```

```
  product_id int primary key,
  product_name varchar2(255) not null unique,
  price number check (price > 0),
  category_id int,
  status varchar2(20) default 'Active',
  constraint fk_prod_cat foreign key (category_id) references Category(category_id)
);
```



It was a complete headache to compile all these outputs because once a command is executed, you can't just re-run it later without it becoming a total mess, and it was a nightmare to clean them up.